

Double-click (or enter) to edit

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import plotly.express
from sklearn.model_selection import train_test_split
import plotly.express as px
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
```

```
file = '/content/Dataset 1.csv'
df = pd.read_csv(file)
```

```
df.isnull().sum()
```

	0
Transaction ID	0
Date	0
Customer ID	0
Gender	0
Age	0
Product Category	0
Quantity	0
Price per Unit	0
Total Amount	0

dtype: int64

```
df.info(), df.head(), df.tail()
```

```
<class 'pandas.core.frame.DataFrame'>
```

```
RangeIndex: 1000 entries, 0 to 999
```

```
Data columns (total 9 columns):
```

#	Column	Non-Null Count	Dtype
0	Transaction ID	1000 non-null	int64
1	Date	1000 non-null	object
2	Customer ID	1000 non-null	object
3	Gender	1000 non-null	object
4	Age	1000 non-null	int64
5	Product Category	1000 non-null	object
6	Quantity	1000 non-null	int64
7	Price per Unit	1000 non-null	int64
8	Total Amount	1000 non-null	int64

```
dtypes: int64(5), object(4)
```

```
memory usage: 70.4+ KB
```

```
(None,
```

	Transaction ID	Date	Customer ID	Gender	Age	Product Category
0	1	2023-11-24	CUST001	Male	34	Beauty
1	2	2023-02-27	CUST002	Female	26	Clothing
2	3	2023-01-13	CUST003	Male	50	Electronics
3	4	2023-05-21	CUST004	Male	37	Clothing
4	5	2023-05-06	CUST005	Male	30	Beauty

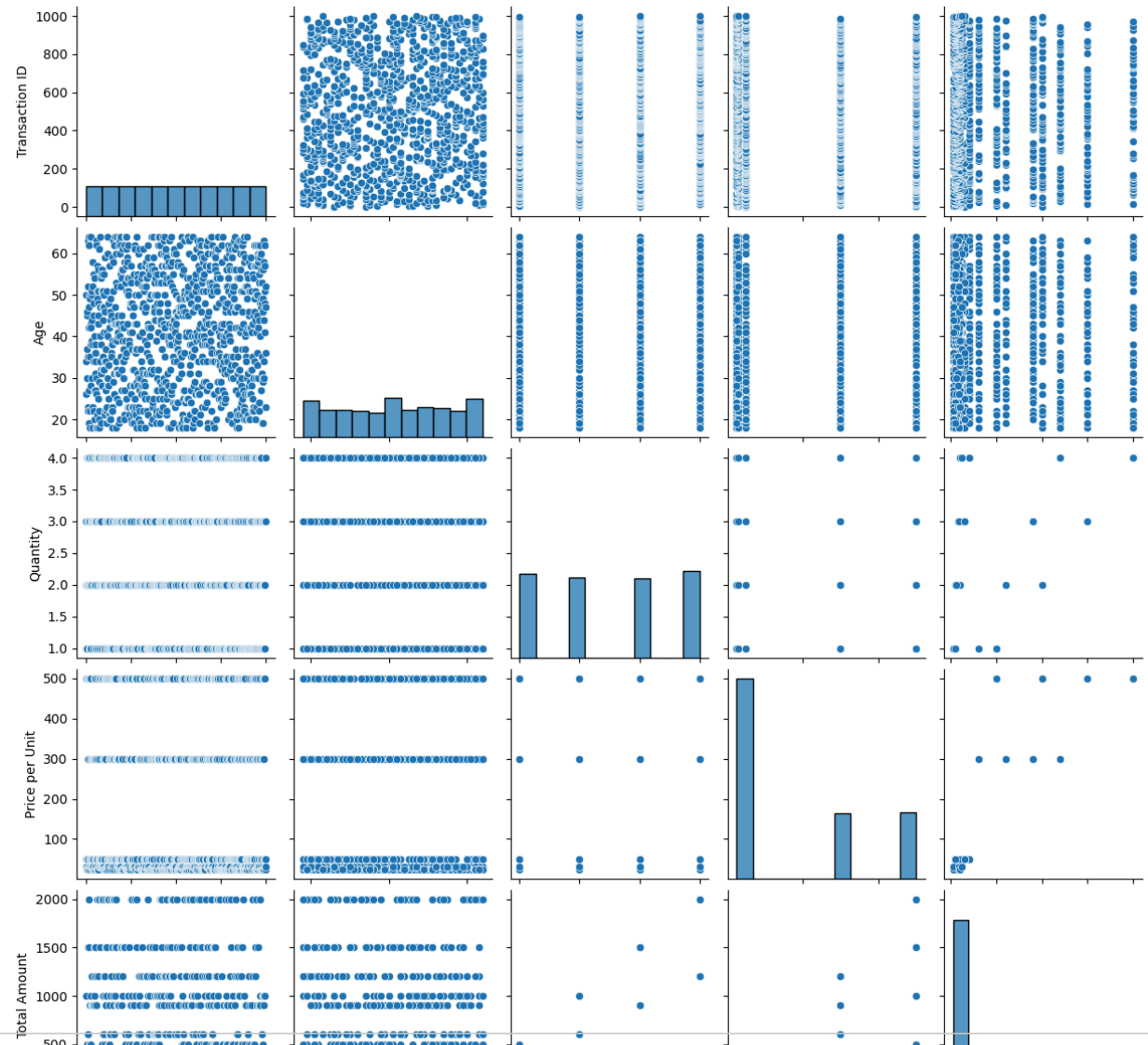
	Quantity	Price per Unit	Total Amount
0	3	50	150
1	2	500	1000
2	1	30	30

	Transaction ID	Date	Customer ID	Gender	Age	Product Category
3	1	500	500			
4	2	50	100			
995	996	2023-05-16	CUST996	Male	62	Clothing
996	997	2023-11-17	CUST997	Male	52	Beauty
997	998	2023-10-29	CUST998	Female	23	Beauty
998	999	2023-12-05	CUST999	Female	36	Electronics
999	1000	2023-04-12	CUST1000	Male	47	Electronics

	Quantity	Price per Unit	Total Amount
995	1	50	50
996	3	30	90
997	4	25	100
998	3	50	150
999	4	30	120

```
sns.pairplot(data=df)
```

```
<seaborn.axisgrid.PairGrid at 0x791251fe2120>
```



```
df['Date'] = pd.to_datetime(df['Date'])
```

```
# Calculate the number of duplicate rows
num_duplicates = df.duplicated().sum()
```

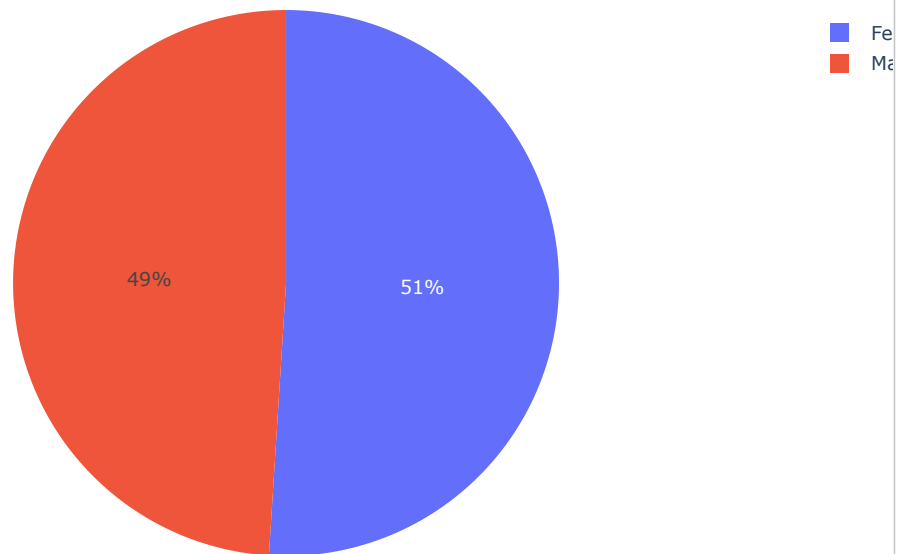
```
if num_duplicates > 0:
    df.drop_duplicates(inplace=True)
    print(f"Dropped {num_duplicates} duplicate rows.")
```

```
# Check for missing values after potential duplicate removal
# 'df.isnull().any().any()' checks if there are any True values (NaNs) in the entire DataFrame
if df.isnull().any().any():
```

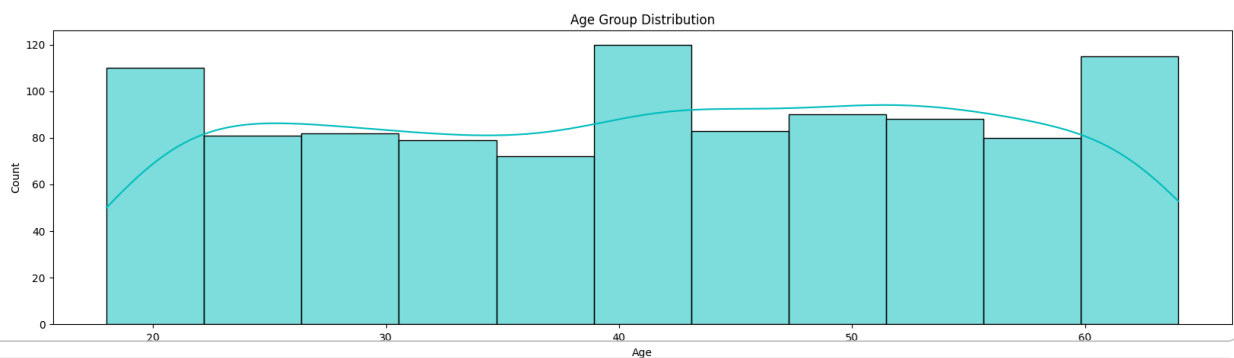
```
df.dropna(inplace=True)
print("Dropped rows with missing value")
```

```
#Pie Chart
#Gender Distribution
df1 = df['Gender']
fig = px.pie(df1,
             names='Gender',
             title='Gender Distribution')
fig.show()
```

Gender Distribution



```
#Histogram Plot
#Age Group Distribution
plt.figure(figsize=(20,5))
sns.histplot(data=df,
             x=df['Age'].sort_values(),
             kde=True,
             color='c')
plt.title('Age Group Distribution')
plt.xticks(rotation=5)
plt.show()
```

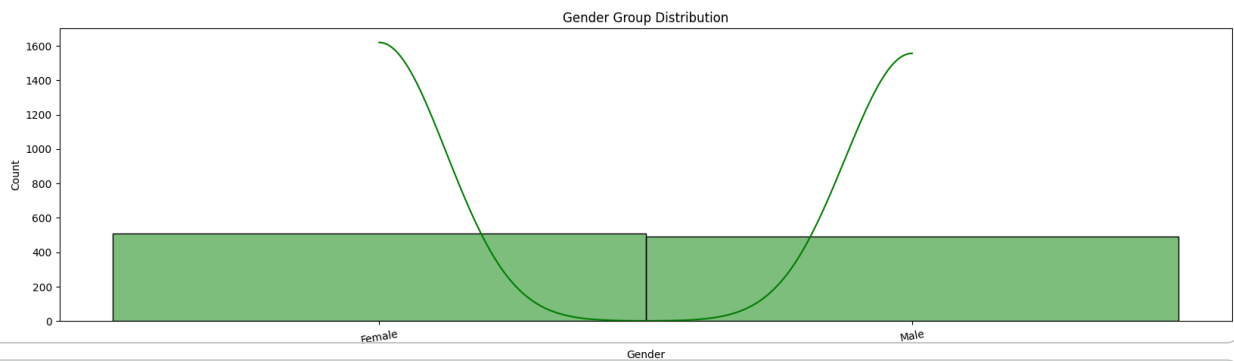


```
#Hist plot
#Gender Group Distribution
plt.figure(figsize=(20,5))
sns.histplot(data=df,
             x=df['Gender'].sort_values(),
```

```

kde=True,
color='g')
plt.title('Gender Group Distribution')
plt.xticks(rotation=10)
plt.show()

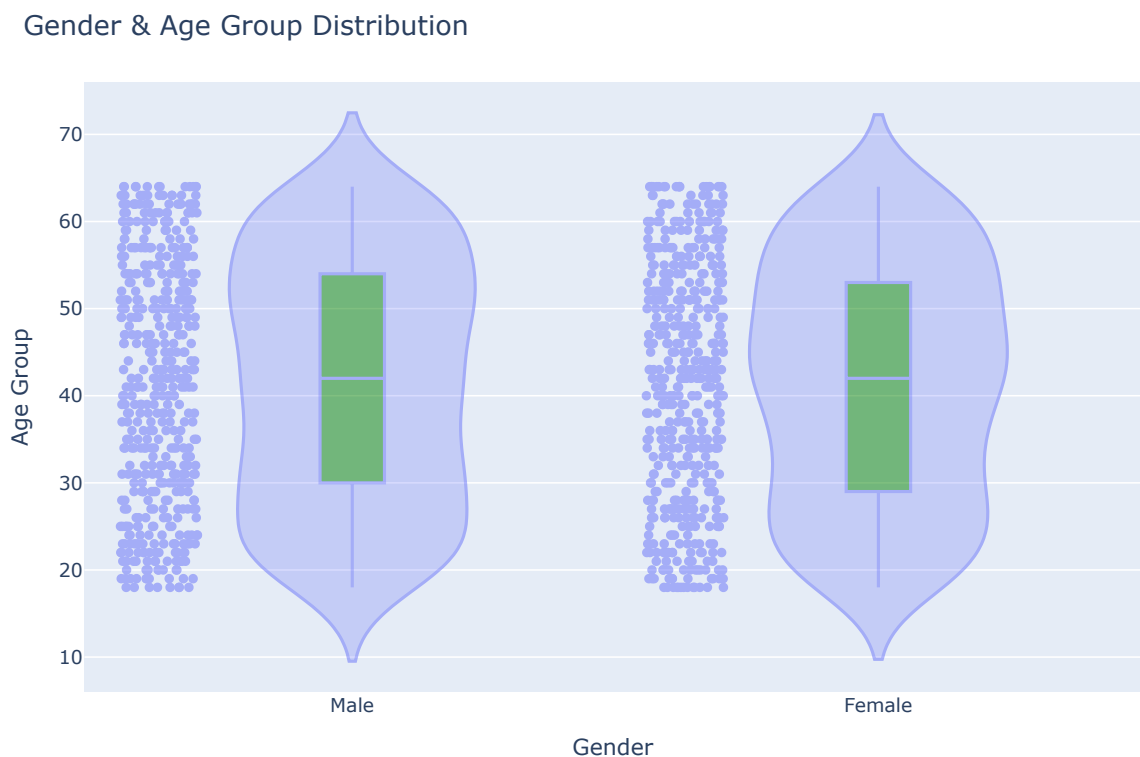
```



```

#Violin plot
#Gender & Age Group Distribution
plot = px.violin(x=df['Gender'],
                 y=df['Age'],
                 box=True,
                 points='all')
plot.update_traces(
    box_fillcolor='green',
    opacity=0.5
)
plot.update_layout(
    title='Gender & Age Group Distribution',
    xaxis_title='Gender',
    yaxis_title='Age Group'
)
plot.show()

```

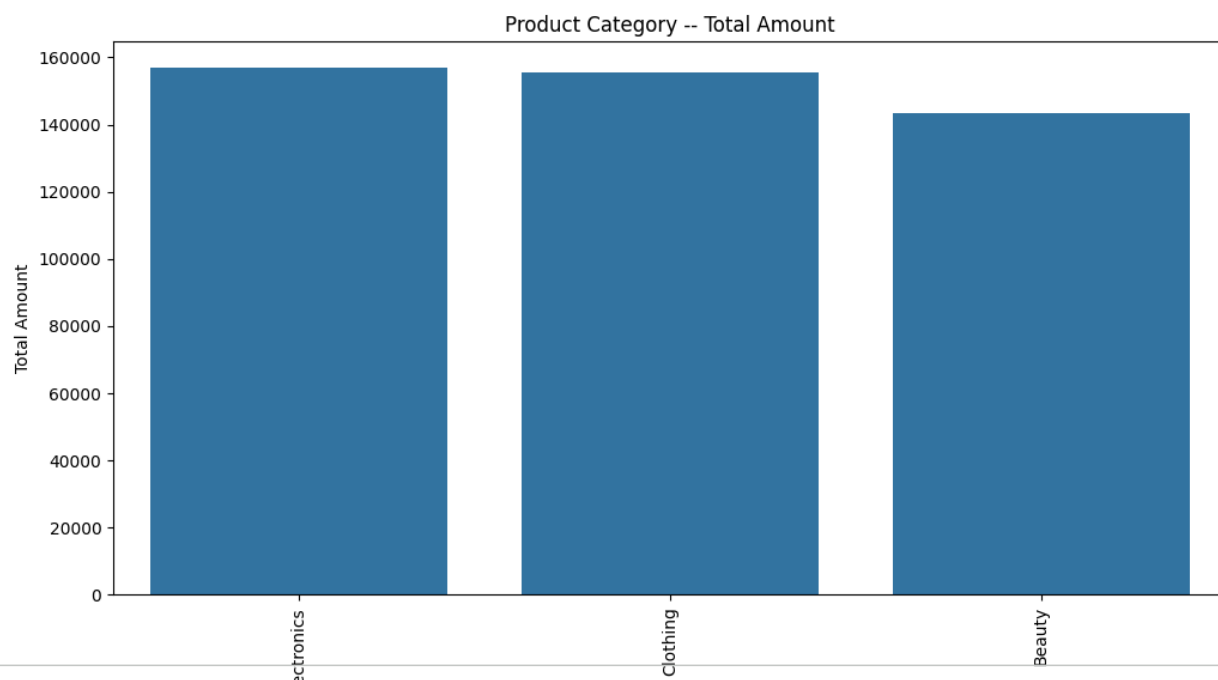


```

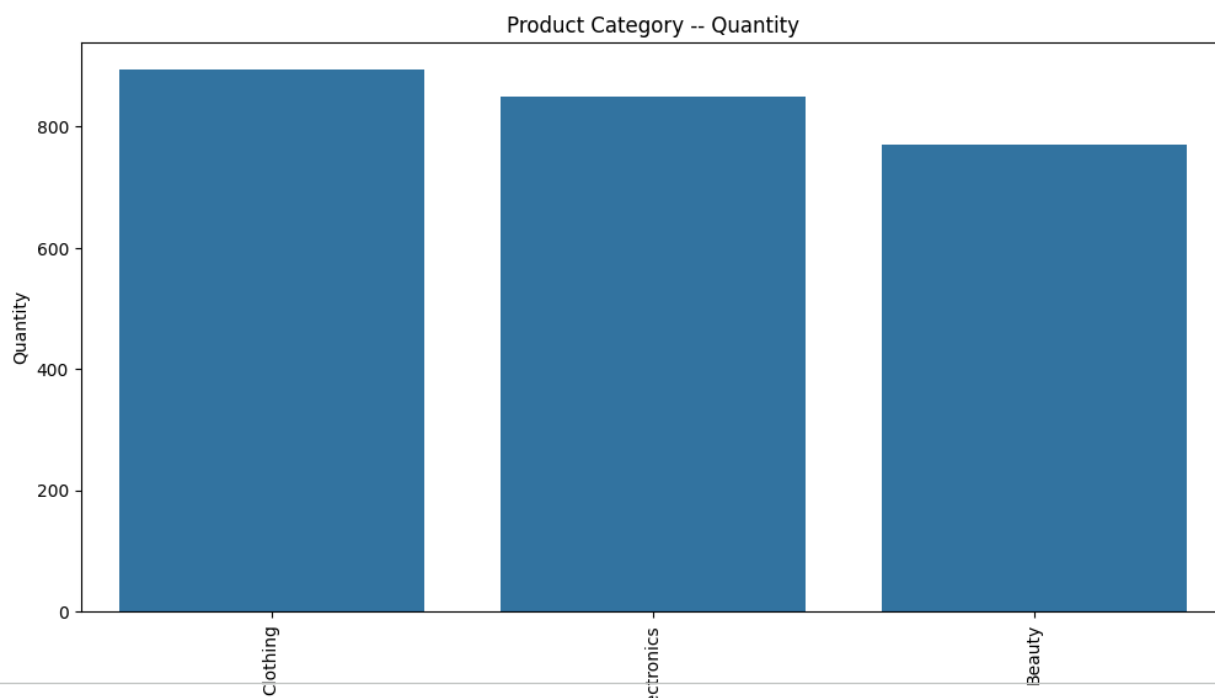
#Bar Plot
#Gold_Item_net_wt_sum
Gold_item_net_wt = df.groupby('Product Category')['Total Amount'].sum().nlargest(40)
plt.figure(figsize=(12, 6))

```

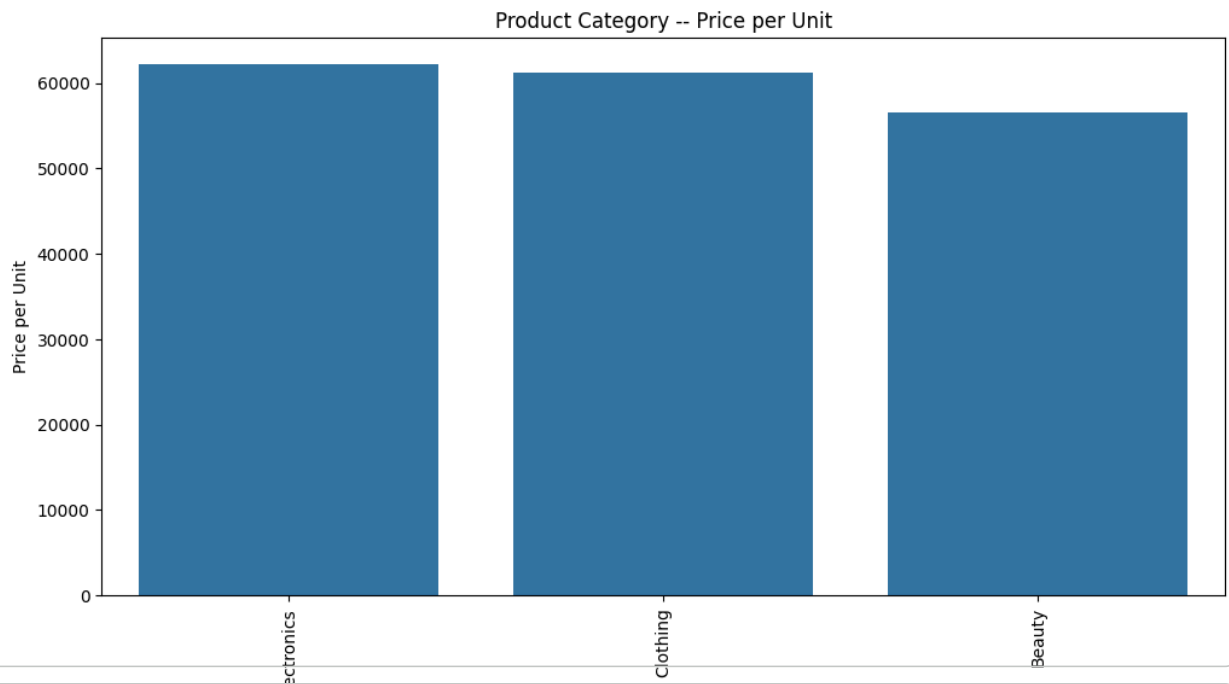
```
sns.barplot(  
    x=Gold_item_net_wt.index,  
    y=Gold_item_net_wt.values)  
plt.xticks(rotation=90)  
plt.title("Product Category -- Total Amount")  
plt.xlabel("Product Category")  
plt.ylabel("Total Amount")  
plt.show()
```



```
#Bar Plot  
#Gold_Item_net_wt_sum  
Gold_item_net_wt = df.groupby('Product Category')['Quantity'].sum().nlargest(40)  
plt.figure(figsize=(12, 6))  
sns.barplot(  
    x=Gold_item_net_wt.index,  
    y=Gold_item_net_wt.values)  
plt.xticks(rotation=90)  
plt.title("Product Category -- Quantity")  
plt.xlabel("Product Category")  
plt.ylabel("Quantity")  
plt.show()
```



```
#Bar Plot
#Gold_Item_net_wt_sum
Gold_item_net_wt = df.groupby('Product Category')['Price per Unit'].sum().nlargest(40)
plt.figure(figsize=(12, 6))
sns.barplot(
    x=Gold_item_net_wt.index,
    y=Gold_item_net_wt.values)
plt.xticks(rotation=90)
plt.title("Product Category -- Price per Unit")
plt.xlabel("Product Category")
plt.ylabel("Price per Unit")
plt.show()
```



```
plt.figure(figsize=(20,6))
sns.lineplot(data=df,x=df['Date'],y=df['Total Amount'],color='c')
plt.show
```

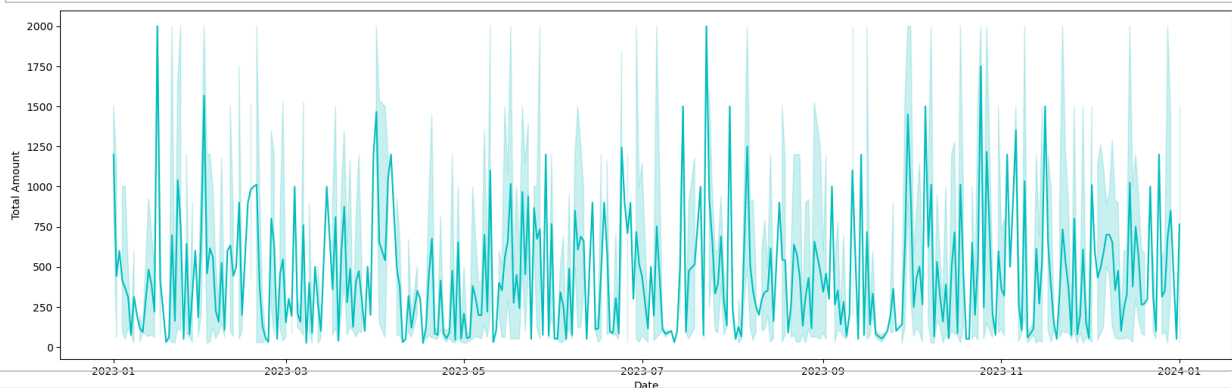
matplotlib.pyplot.show
def show(*args, **kwargs) -> None

Display all open figures.

Parameters

block : bool, optional

Whether to wait for all figures to be closed before returning.



```
plt.figure(figsize=(20,6))
sns.lineplot(data=df,x=df['Date'],y=df['Quantity'],color='c')
plt.show
```

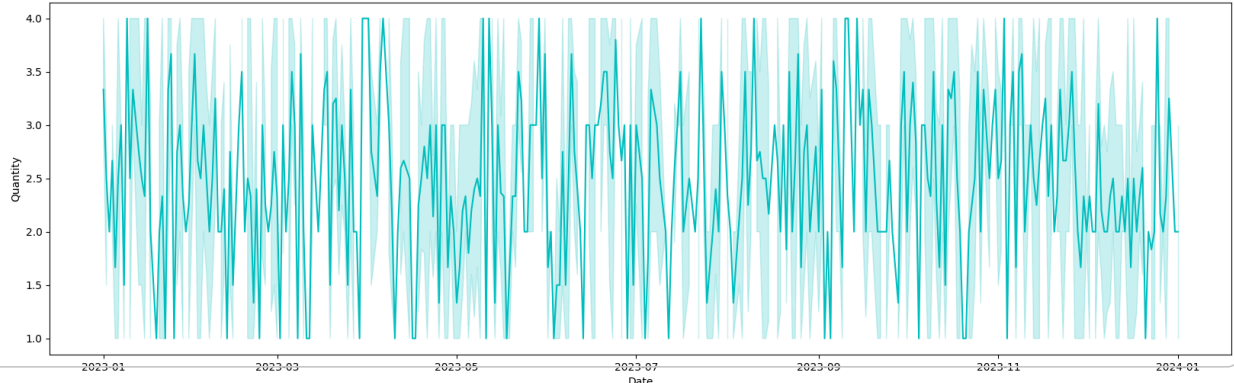
```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

Display all open figures.

Parameters

block : bool, optional

Whether to wait for all figures to be closed before returning.



```
plt.figure(figsize=(20,6))
sns.lineplot(data=df,x=df['Date'],y=df['Price per Unit'],color='c')
plt.show
```

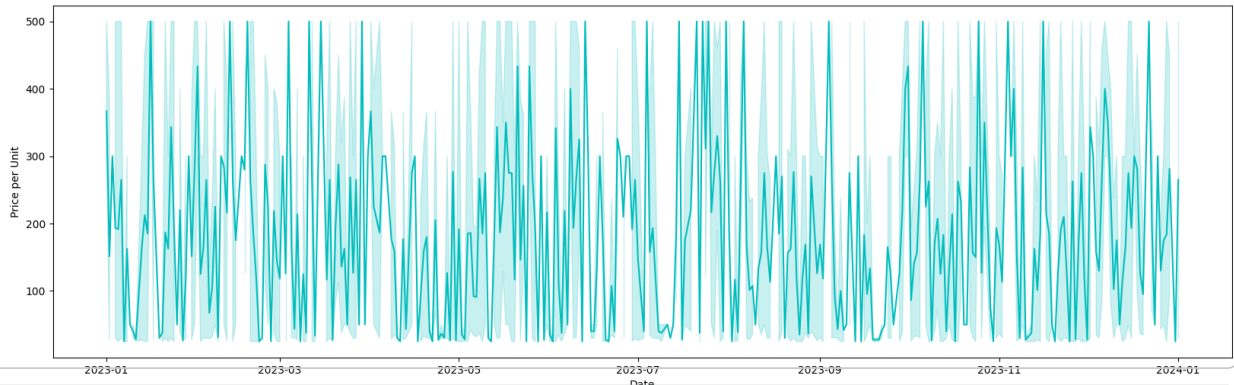
```
matplotlib.pyplot.show
def show(*args, **kwargs) -> None
```

Display all open figures.

Parameters

block : bool, optional

Whether to wait for all figures to be closed before returning.



```
X=df.drop(['Date','Customer ID'],axis=1)
y=df['Total Amount']
#Split the dataset into train and test data
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
#We are getting cat columns and num columns for extra encoding purpose
num_cols=[col for col in X_train.columns if X_train[col].dtype!='0']
cat_cols=[col for col in X_train.columns if X_train[col].dtype=='0']
```

```
#numerical processing pipeline
numeric_processor=Pipeline(
    steps=[
        ("scaler",StandardScaler())
    ]
)

#categorical processing
from sklearn.preprocessing import OneHotEncoder
categorical_processor=Pipeline(
    steps=[
```

```
("onehot",OneHotEncoder(handle_unknown='ignore'))]  
)
```